



ระบบจัดการและติดตามดูแลต้นไม้ PlantPal

จัดทำโดย

นางสาวกมลพร	เกตุแก้ว	รหัสนักศึกษา 673380571-1
นางสาวพริ้มภัทร	ภาวิฒนคุณ	รหัสนักศึกษา 673380594-9
นางสาวมุกดา	บุญประจันต์	รหัสนักศึกษา 673380598-1
นางสาวสรนันท์	บุสดี	รหัสนักศึกษา 673380605-0

เสนอ

รศ.ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของวิชา CP353002

หลักการออกแบบพัฒนาซอฟต์แวร์ ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์มหาวิทยาลัยขอนแก่น

สารบัญ

บทที่ 1 บทนำ	3
1.1 ที่มาและปัญหา.....	3
1.2 ขอบเขตและผู้ใช้ระบบ	3
1.3 สมาชิกและหน้าที่.....	4
บทที่ 2 การออกแบบระบบ.....	5
2.1 สถาปัตยกรรมระบบ.....	5
2.2 Use Case	6
2.3 Diagram การออกแบบ	7
บทที่ 3 ฐานข้อมูล.....	10
3.1 ER Diagram	10
3.2 Data Dictionary	11
บทที่ 4 หลักการออกแบบโค้ด.....	12
4.1 SOLID Principles	12
4.2 Design Patterns.....	12
บทที่ 5 การพัฒนาและทดสอบ	14
5.1 REST API และ Swagger.....	14
5.2 การทดสอบ	14
5.3 Deployment และ CI/CD.....	15
บทที่ 6 สรุป.....	16
6.1 ปัญหาที่พบ	16
6.2 แนวทางปรับปรุง.....	16

บทที่ 1

บทนำ

1.1 ที่มาและปัญหา

ปัจจุบันคนจำนวนมากปลูกต้นไม้ในบ้าน คอนโด และที่ทำงาน แต่ต้นไม้แต่ละพันธุ์ต้องการการดูแลไม่เหมือนกัน บางพันธุ์ต้องรดน้ำทุก 3 วัน บางพันธุ์ทุก 3 สัปดาห์ และยังมีรอบใส่ปุ๋ยและเปลี่ยนกระถางของตัวเอง เมื่อปลูกหลายต้นพร้อมกัน ผู้ปลูกจึงมักจำไม่ได้ว่าต้นไหนดูแลไปแล้วเมื่อไร ต้นไหนถึงเวลาต้องดูแล บันทึกการดูแลก็กระจายอยู่ในสมุดหรือนोटในโทรศัพท์ ย้อนดูประวัติไม่ได้ และเมื่อต้นไม้มีอาการผิดปกติ เช่น ใบเหลืองหรือใบร่วง ก็ไม่รู้ว่าจะถามใครและติดตามผลอย่างไร

ทีมจึงพัฒนา PlantPal (Tree Management and Care Tracking System) เว็บแอปพลิเคชันสำหรับจัดการข้อมูลต้นไม้และติดตามการดูแลอย่างเป็นระบบ เมื่อผู้ใช้เพิ่มต้นไม้ ระบบจะสร้างตารางรดน้ำ ใส่ปุ๋ย และเปลี่ยนกระถางให้อัตโนมัติตามพันธุ์ไม้ ผู้ใช้กด "ทำแล้ว" เพื่อบันทึกการดูแล ดูปฏิทินล่วงหน้าและประวัติย้อนหลังได้ ส่งรายงานสุขภาพต้นไม้พร้อมรูปให้ผู้ดูแลระบบแนะนำ และได้รับการแจ้งเตือนงานดูแลทุกเช้า นอกจากนี้โครงงานยังมีเป้าหมายเพื่อฝึกออกแบบซอฟต์แวร์ตามหลัก SOLID และนำ Design Patterns มาแก้ปัญหาจริงในระบบ

1.2 ขอบเขตและผู้ใช้ระบบ

ระบบมีผู้ใช้ 4 กลุ่ม ผู้เยี่ยมชม สมัครสมาชิกและเข้าสู่ระบบได้ ผู้ใช้ (USER) จัดการข้อมูลของตัวเองได้ทั้งหมด และเห็นเฉพาะข้อมูลของตัวเอง ผู้ดูแลระบบ (ADMIN) ทำได้ทุกอย่างที่ผู้ใช้ทำได้ และจัดการระบบผ่านหน้า /admin ส่วน ระบบตั้งเวลา เป็นงานที่รันเองทุกเช้าตามเวลาไทย

ขอบเขตของระบบมีดังนี้

1. **ผู้ใช้** สมัครสมาชิก เข้าสู่ระบบ แก้ไขโปรไฟล์ ดูแดชบอร์ดสรุป เพิ่ม แก้ไข ลบต้นไม้และย้อนการแก้ไขได้ เปลี่ยนสถานะสุขภาพต้นไม้ ดูตารางการดูแลแบ่งเป็น เลย์กำหนด / วันนี้ / ถัดไป กด "ทำแล้ว" ดูปฏิทิน 30 วันและประวัติการดูแล ส่งรายงานสุขภาพพร้อมรูป และรับการแจ้งเตือน
1. **ผู้ดูแลระบบ** ตอบคำถามหรือปฏิเสธรายงานสุขภาพ จัดการพันธุ์ไม้และรอบการดูแล และจัดการผู้ใช้ (เปลี่ยนบทบาท ระงับบัญชี และย้อนคำสั่งได้)
2. **ระบบตั้งเวลา** 08:00 แจ้งเตือนงานที่ครบกำหนดพรุ่งนี้ 08:05 แจ้งเตือนงานที่เลยกำหนด และ 08:10 ปิดรายงานที่ไม่มีการอัปเดตผลเกิน 14 วัน
3. **REST API** สำหรับต้นไม้และรายงานสุขภาพ พร้อมเอกสาร Swagger

สิ่งที่อยู่นอกขอบเขต ได้แก่ แอปมือถือแบบติดตั้ง การแจ้งเตือนผ่านอีเมลหรือ LINE และการแนบรูปผ่าน REST API

1.3 สมาชิกและหน้าที่

ตารางที่ 1.1 สมาชิกกลุ่มและหน้าที่รับผิดชอบ

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา	Section	Branch	หน้าที่รับผิดชอบ
1	นางสาวกมลพร เกตุแก้ว	673380571-1	3	Kamolpon_6733805711_03	รายงานสุขภาพ การแจ้งเตือน งานตั้งเวลา
2	นางสาวพริ้มภัทร ภาวิฒนคุณ	673380594-9	3	preemphat_6733805949_03	สมัครสมาชิก เข้าสู่ระบบ จัดการผู้ใช้
3	นางสาวมุกดา บุญประจันทร์	673380598-1	3	Mukda_6733805981_03	จัดการต้นไม้ สถานะสุขภาพ REST API
4	นางสาวสรนันท์ บุสดี	673380605-0	3	Soranan_6733806050_03	ตารางการดูแล ปฏิทิน ประวัติ การทดสอบ

ทีมพัฒนาร่วมกันบน GitHub แต่ละคนทำงานบน branch ของตัวเอง แล้วเปิด Pull Request เข้า branch develop ให้เพื่อนตรวจสอบก่อน merge ทุกครั้ง

บทที่ 2

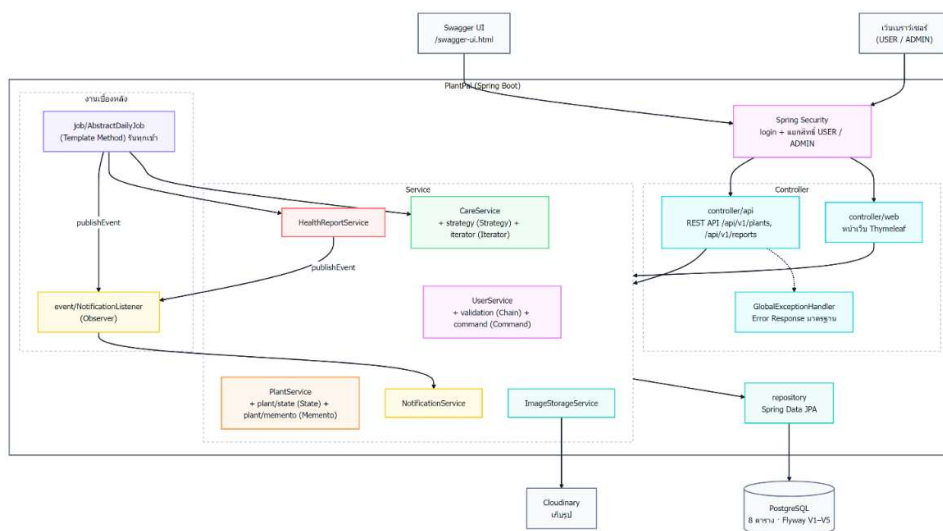
การออกแบบระบบ

2.1 สถาปัตยกรรมระบบ

Tech Stack ระบบพัฒนาด้วย Java 21 และ Spring Boot 4.1.1 (Web MVC, Data JPA, Security, Validation) หน้าเว็บใช้ Thymeleaf ฐานข้อมูลเป็น PostgreSQL บน Neon จัดการ schema ด้วย Flyway เก็บรูปภาพบน Cloudinary สร้างเอกสาร API ด้วย springdoc-openapi (Swagger) ทดสอบด้วย JUnit 5 และ Mockito และ build / deploy ด้วย Maven, Docker, GitHub Actions และ Render

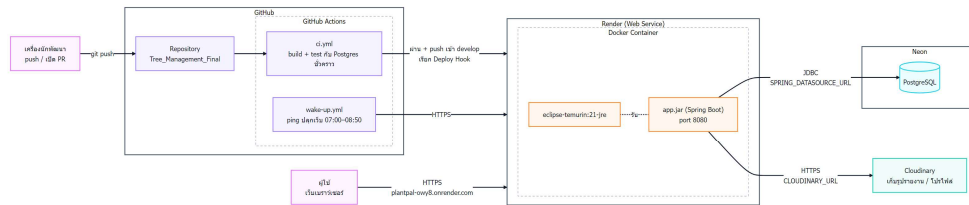
Layered Architecture โค้ดแบ่งเป็นชั้นที่เรียกกันได้ว่าทางเดียวจากบนลงล่าง ทุก request ผ่าน Spring Security ก่อนเพื่อตรวจสอบการเข้าสู่ระบบและสิทธิ์ จากนั้น Controller รับ request (หน้าเว็บส่งข้อมูลให้ Thymeleaf ส่วน REST API ตอบ JSON) แล้วเรียก Service ที่เก็บ business logic และ transaction ทั้งหมด Service อ่านเขียนฐานข้อมูลผ่าน Repository และ Domain เก็บ entity กับ enum Controller ไม่เรียก Repository โดยตรง หน้าเว็บกับ REST API จึงใช้ Service ตัวเดียวกันและกติกายู่ที่เดียว

Component นอกจากชั้นหลักแล้ว ระบบยังมีงานเบื้องหลังแบ่งเป็น 2 ส่วน คือ ส่วนที่หนึ่ง NotificationListener ที่รับ event แล้วสร้างการแจ้งเตือน และ Job รายวันที่รันเองทุกเช้า ส่วนระบบภายนอกคือ PostgreSQL และ Cloudinary



รูปที่ 2.1 Component Diagram

และส่วนที่สอง **Deployment** โค้ดอยู่บน GitHub เมื่อ push เข้า develop GitHub Actions จะ build และรันเทสต์ ถ้าผ่านจะสั่ง Render สร้าง Docker container ใหม่ แอปเชื่อมต่อ Neon (ฐานข้อมูล) และ Cloudinary (รูปภาพ) ผ่าน environment variable

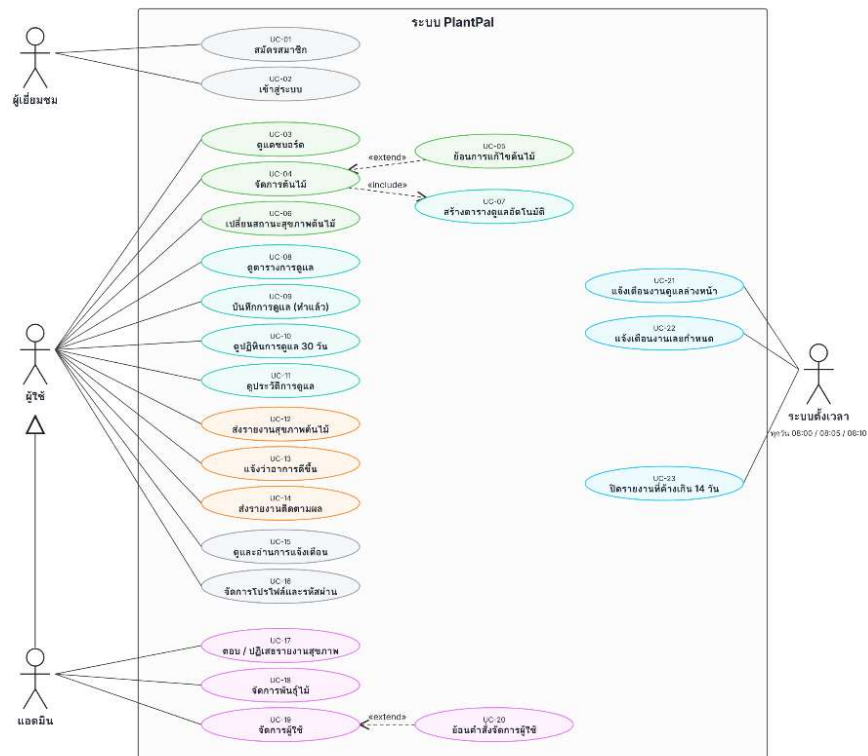


รูปที่ 2.2 Deployment Diagram

2.2 Use Case

ระบบมี 23 Use Case แบ่งตามผู้ใช้ได้ดังนี้

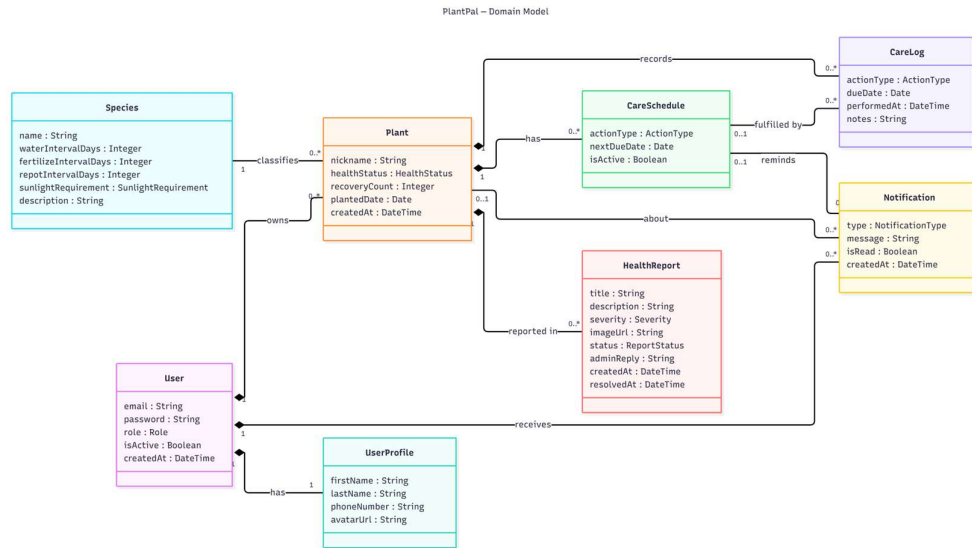
1. **ผู้เยี่ยมชม** สมัครสมาชิก (ตรวจสอบรูปแบบอีเมล อีเมลซ้ำ ความยาวและการยืนยันรหัสผ่าน) และเข้าสู่ระบบ (บัญชีที่ถูกระงับเข้าไม่ได้)
2. **ผู้ใช้** ดูแดชบอร์ด จัดการต้นไม้ ซึ่ง «include» การสร้างตารางดูแลอัตโนมัติ และถูก «extend» ด้วยการย้อนการแก้ไข เปลี่ยนสถานะสุขภาพ ดูตารางการดูแล บันทึกการดูแล ดูปฏิทินและประวัติ ส่งรายงานสุขภาพ แจ้งว่าอาการดีขึ้น ส่งรายงานติดตามผล ดูการแจ้งเตือน และจัดการโปรไฟล์
3. **ผู้ดูแลระบบ** สืบทอดจากผู้ใช้ และเพิ่ม ตอบ / ปฏิเสธรายงานสุขภาพ จัดการพันธุ์ไม้ และจัดการผู้ใช้ ซึ่งถูก «extend» ด้วยการย้อนคำสั่ง
4. **ระบบตั้งเวลา** แจ้งเตือนงานดูแลล่วงหน้า แจ้งเตือนงานเลยกำหนด และปิดรายงานที่ค้างเกิน 14 วัน



รูปที่ 2.3 Use Case Diagram

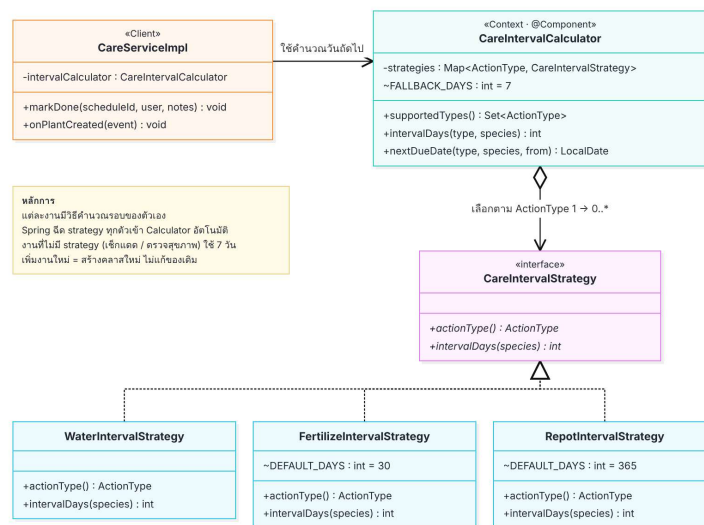
2.3 Diagram การออกแบบ

Domain Model ระบบมีข้อมูลหลัก 8 อย่าง ผู้ใช้ (User) 1 คนมีโปรไฟล์ (UserProfile) 1 อัน มีต้นไม้ (Plant) ได้หลายต้น และได้รับการแจ้งเตือน (Notification) หลายรายการ ต้นไม้แต่ละต้นอยู่ในสายพันธุ์ (Species) หนึ่ง ซึ่งเก็บรอบการดูแลไว้ ต้นไม้แต่ละต้นมีตารางดูแล (CareSchedule) ได้หลายงาน มีประวัติการดูแล (CareLog) และรายงานสุขภาพ (HealthReport) ได้หลายรายการ



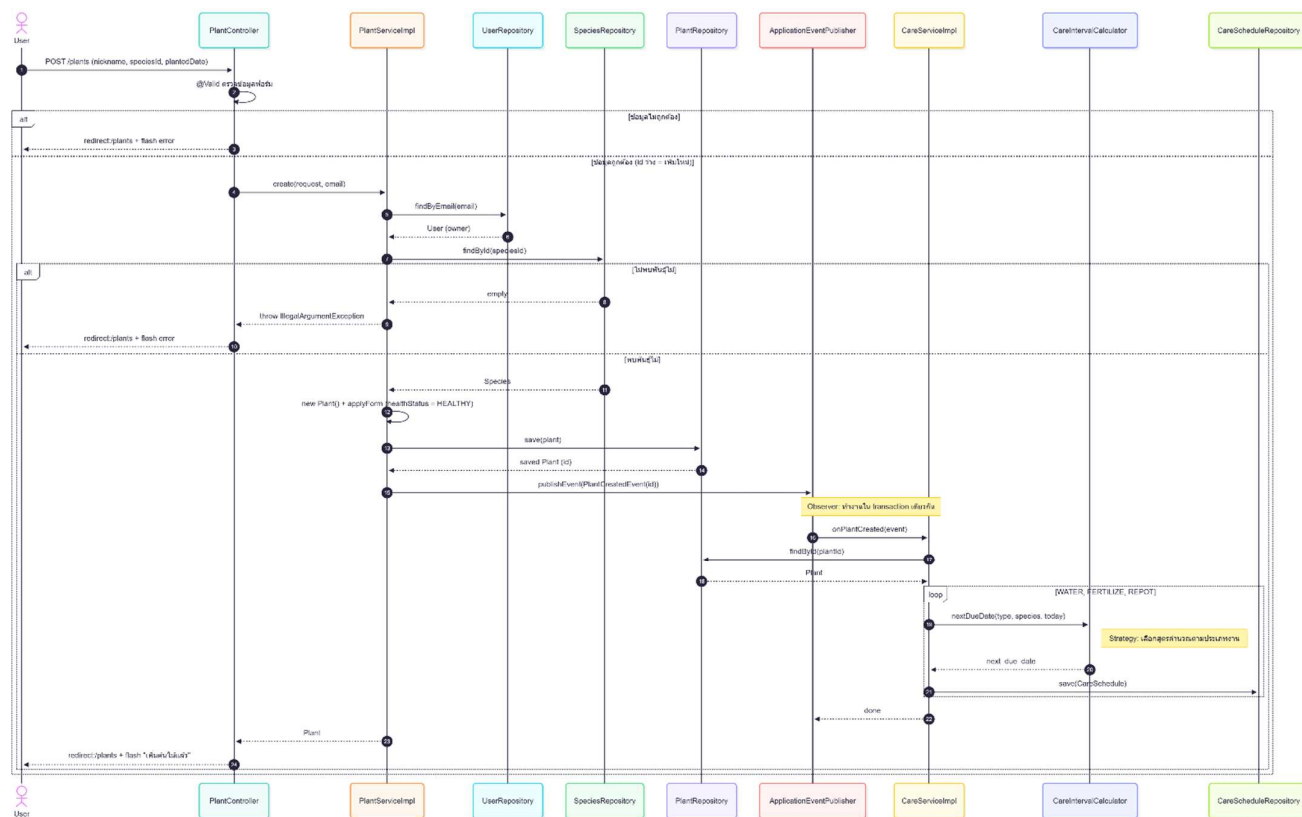
รูปที่ 2.4 Domain Model

Class Diagram ทีมแยก Class Diagram ตาม Design Pattern ที่ใช้ เพื่อให้เห็นความสัมพันธ์ของคลาสในแต่ละส่วนชัดเจน ตัวอย่างเช่น Strategy ที่ใช้คำนวณรอบการดูแล CareServiceImpl เรียก CareIntervalCalculator ซึ่งเลือก strategy ของแต่ละงานผ่าน interface CareIntervalStrategy



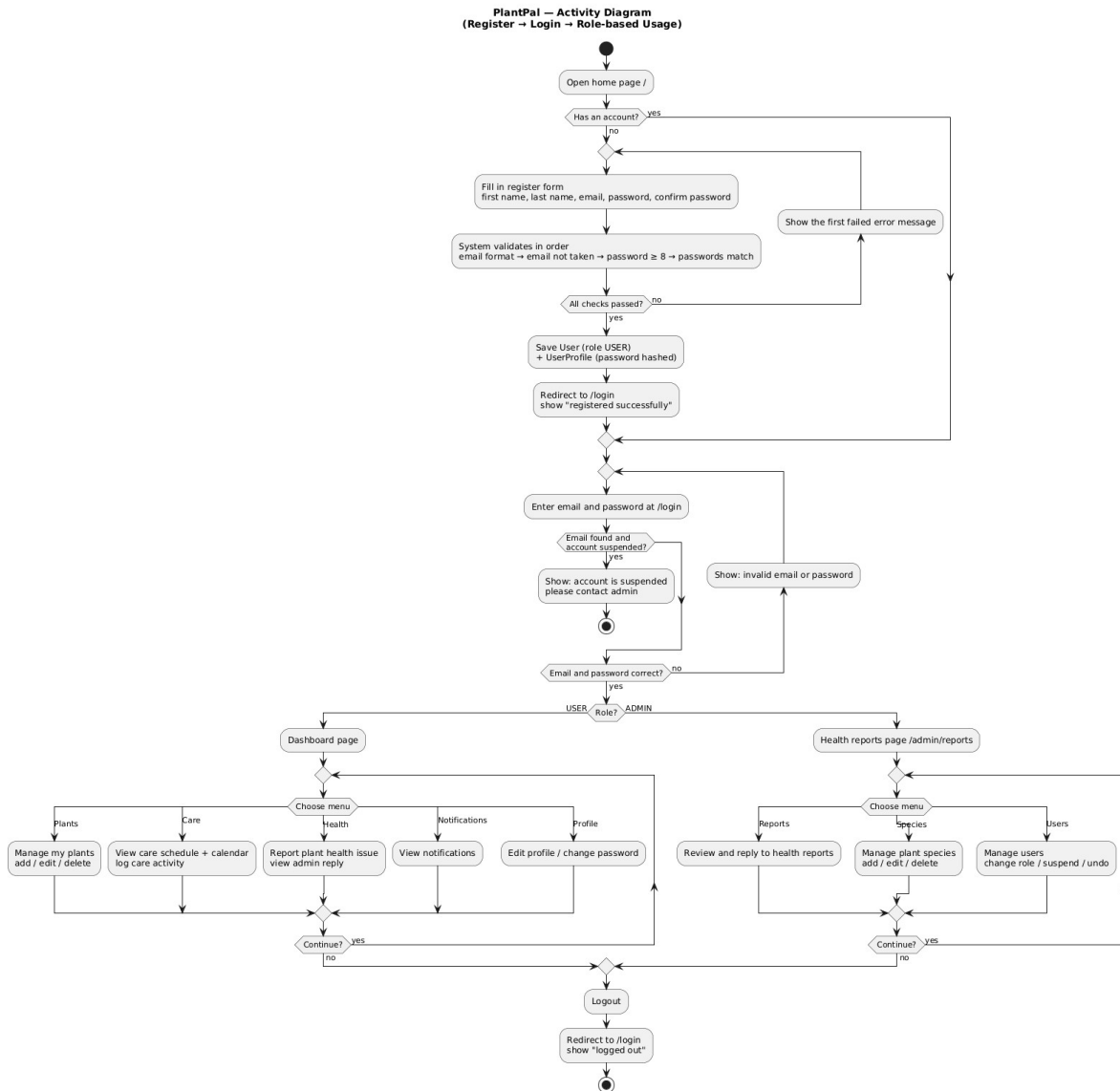
รูปที่ 2.5 Class Diagram: Strategy

Sequence Diagram ทีมทำ Sequence Diagram 5 scenario คือ สมัครสมาชิก เข้าสู่ระบบ เพิ่มต้นไม้ แก้ปัญหาสุขภาพแล้วแอดมินตอบ และบันทึกการดูแล ตัวอย่างการเพิ่มต้นไม้ PlantController ส่งข้อมูลให้ PlantServiceImpl บันทึกต้นไม้แล้วประกาศ PlantCreatedEvent จากนั้น CareServiceImpl รับ event แล้วสร้างตารางดูแล 3 งาน โดยให้ CareIntervalCalculator คำนวณวันครบกำหนดตามพันธุ์ไม้ ทั้งหมดอยู่ใน transaction เดียว



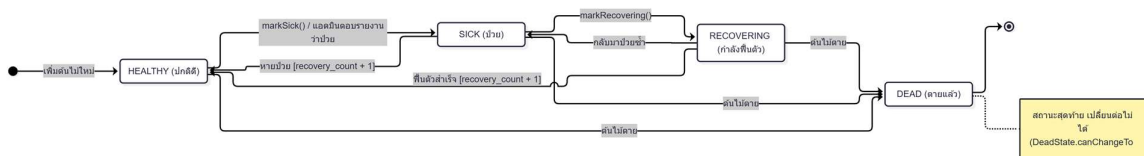
รูปที่ 2.6 Sequence Diagram: เพิ่มต้นไม้

Activity Diagram ผู้ใช้เริ่มจากสมัครสมาชิกและเข้าสู่ระบบ ระบบตรวจว่าบัญชีถูกระงับหรือไม่ แล้วแยกหน้าตามบทบาท ผู้ใช้ไปหน้าแดชบอร์ดเพื่อจัดการต้นไม้ ดูแลต้นไม้ และแก้ปัญหสุขภาพ ส่วนแอดมินไปหน้ารายงานสุขภาพเพื่อตอบรายงาน จัดการพันธุ์ไม้ และจัดการผู้ใช้ จนออกจากระบบ



รูปที่ 2.7 Activity Diagram

State Diagram สถานะสุขภาพต้นไม้มี 4 สถานะ HEALTHY เปลี่ยนเป็น SICK หรือ DEAD ได้ SICK เปลี่ยนเป็น RECOVERING, HEALTHY หรือ DEAD ได้ RECOVERING เปลี่ยนเป็น HEALTHY, SICK หรือ DEAD ได้ ส่วน DEAD เปลี่ยนต่อไม่ได้อีก ทุกครั้งที่ต้นไม้กลับมาเป็น HEALTHY ระบบนับจำนวนครั้งที่ฟื้นตัวเพิ่ม 1



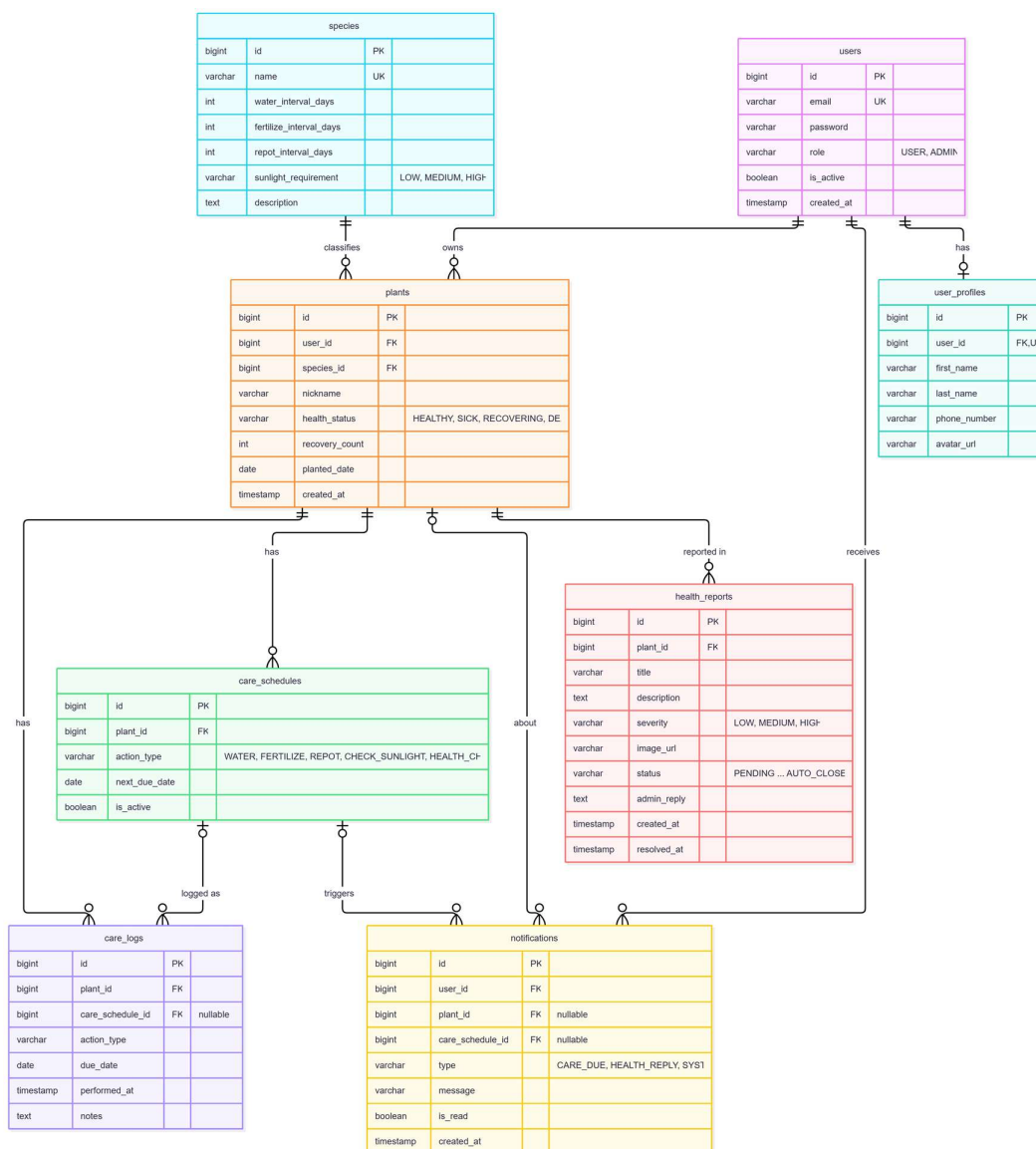
รูปที่ 2.8 State Diagram สถานะสุขภาพต้นไม้

บทที่ 3

ฐานข้อมูล

3.1 ER Diagram

ระบบใช้ PostgreSQL บน Neon มี 8 ตาราง จัดการ schema ด้วย Flyway ซึ่งสร้างตารางและข้อมูลตัวอย่างให้อัตโนมัติตอนแอปเริ่มทำงาน ความสัมพันธ์หลักคือ users มี user_profiles 1 แถว และมี plants กับ notifications ได้หลายแถว plants อ้างถึง species และมี care_schedules, care_logs, health_reports ได้หลายแถว เมื่อลบผู้ใช้หรือต้นไม้ ข้อมูลลูกจะถูกลบตาม (CASCADE) แต่ลบสายพันธุ์ที่ยังมีต้นไม้ใช้อยู่ไม่ได้ (RESTRICT)



รูปที่ 3.1 ER Diagram

3.2 Data Dictionary

ทุกตารางใช้ id ชนิด BIGINT เป็น Primary Key รายละเอียดแต่ละตารางมีดังนี้

4. **users** บัญชีผู้ใช้ เก็บ email (ไม่ซ้ำ), password ที่เข้ารหัสด้วย BCrypt, role (USER / ADMIN), is_active (บัญชีถูกระงับหรือไม่) และ created_at
5. **user_profiles** ข้อมูลส่วนตัว เก็บ user_id (FK ไม่ซ้ำ), first_name, last_name, phone_number และ avatar_url
6. **species** สายพันธุ์ เก็บ name (ไม่ซ้ำ), water_interval_days (บังคับกรอก), fertilize_interval_days และ repot_interval_days (ถ้าว่างใช้ค่าตั้งต้น 30 และ 365 วัน), sunlight_requirement และ description
7. **plants** ต้นไม้ เก็บ user_id, species_id, nickname, health_status (ค่าเริ่มต้น HEALTHY), recovery_count, planted_date และ created_at
8. **care_schedules** ตารางดูแล เก็บ plant_id, action_type (WATER, FERTILIZE, REPOT, CHECK_SUNLIGHT, HEALTH_CHECK), next_due_date และ is_active
ต้นหนึ่งมีงานแต่ละประเภทได้แค่ 1 แถว
9. **care_logs** ประวัติการดูแล เก็บ plant_id, care_schedule_id, action_type, due_date (วันครบกำหนดของรอบนั้น), performed_at (เวลาที่ทำจริง) และ notes
10. **health_reports** รายงานสุขภาพ เก็บ plant_id, title, description, severity, image_url, status (PENDING, IN_PROGRESS, RESOLVED, REJECTED, FOLLOWED_UP, AUTO_CLOSED), admin_reply, created_at และ resolved_at
11. **notifications** การแจ้งเตือน เก็บ user_id, plant_id, care_schedule_id, type (CARE_DUE, HEALTH_REPLY, SYSTEM), message, is_read และ created_at

Schema ถูกสร้างด้วย Flyway migration 5 ไฟล์ V1 สร้างตารางทั้งหมด V2 เพิ่มรอบใส่ปุ๋ยและเปลี่ยนกระถาง V3 เพิ่มผู้ใช้ทดสอบ V4 เพิ่มสถานะรายงาน และ V5 เพิ่มข้อมูลตัวอย่างสำหรับสถิติ

บทที่ 4

หลักการออกแบบโค้ด

4.1 SOLID Principles

1. **Single Responsibility** แต่ละคลาสมีหน้าที่เดียว เช่น UserServiceImpl ทำแค่ขั้นตอนสมัครสมาชิก การตรวจสอบข้อมูลแยกไปอยู่ใน validator 4 ตัว ตัวละเรื่อง การแปลง entity เป็น DTO อยู่ใน mapper และการตอบ error ของ API อยู่ที่ GlobalExceptionHandler ที่เดียว
2. **Open/Closed** เพิ่มความสามารถใหม่ด้วยการเพิ่มคลาส ไม่ต้องแก้ไขโค้ดเดิม เช่น เพิ่มงานดูแลประเภทใหม่แค่สร้าง strategy ใหม่ เพิ่มเงื่อนไขการสมัครแค่สร้าง validator ใหม่ และเพิ่มช่องทางแจ้งเตือนแค่เพิ่ม listener ใหม่
3. **Liskov Substitution** คลาสลูกใช้แทนกันได้โดยระบบยังทำงานถูก เช่น validator ทุกตัวโยน exception แบบเดียวกันเมื่อไม่ผ่าน state ทุกตัวโยน InvalidHealthTransitionException เมื่อเปลี่ยนไม่ได้ และทั้งโปรเจกต์ไม่มีคลาสไหนโยน UnsupportedOperationException
4. **Interface Segregation** แยก interface ให้เล็กตามผู้ใช้งาน เช่น งานเกี่ยวกับผู้ใช้แยกเป็น UserService, ProfileService, AdminUserService และ CurrentUserService คลาสที่อยากรู้แค่ว่าใครเข้าสู่ระบบอยู่จึงไม่ต้องรู้จัก method ของแอดมิน
5. **Dependency Inversion** คลาสขึ้นกับ interface และรับ dependency ผ่าน constructor (@RequiredArgsConstructor กับ field final) ทั้งโปรเจกต์ไม่มี @Autowired และ PlantServiceImpl ประกาศ event แทนการเรียก CareServiceImpl ตรง ๆ โมดูลจึงไม่ผูกกัน

4.2 Design Patterns

Architectural Patterns ระบบใช้ Layered Architecture และ MVC แยกหน้าที่ของแต่ละชั้น ใช้ Repository (Spring Data JPA) แทนการเขียน SQL กระจายใน service ใช้ Service Layer ให้นำเว็บกับ REST API ใช้กติกาเดียวกัน ใช้ DTO และ Mapper ไม่ให้ entity หลุดออกไปทาง API และใช้ Dependency Injection ผ่าน constructor ทำให้สลับ implementation และ mock ตอนทดสอบได้

GoF Behavioral Patterns ทีมใช้ 8 pattern แต่ละตัวแก้ปัญหาจริงในระบบดังนี้

1. **Strategy (สตรันท์)** งานดูแลแต่ละประเภทคำนวณรอบไม่เหมือนกัน จึงแยกเป็น WaterIntervalStrategy, FertilizeIntervalStrategy, RepotIntervalStrategy และให้ CareIntervalCalculator เลือกตามประเภทงาน งานที่ยังไม่มี strategy ใช้รอบสำรอง 7 วัน
2. **Iterator (สตรันท์)** ปฏิทินต้องแสดงครบ 30 วันแม้วันที่ไม่มีงาน CareCalendarIterator จึงคืน "วัน" ที่ละวันผ่าน hasNext() / next() หน้าเว็บไม่ต้องคำนวณวันที่เอง

3. **State (มุกตา)** สถานะสุขภาพเปลี่ยนได้เฉพาะบางเส้นทาง
แต่ละสถานะเป็นคลาสของตัวเองที่ตัดสินใจว่าเปลี่ยนไปไหนได้ กติกาจึงอยู่ที่เดียว
และหน้าเว็บแสดงเฉพาะปุ่มที่กดได้
4. **Memento (มุกตา)** ก่อนแก้ไขต้นไม้ ระบบเก็บ PlantSnapshot ของค่าเดิมไว้ใน PlantEditHistory
ผู้ใช้จึงย้อนการแก้ไขล่าสุดได้โดยไม่ต้องสร้างตารางเพิ่ม
5. **Observer (กมลพร)** หลายโมดูลต้องสร้างการแจ้งเตือน จึงให้ประกาศ event แล้ว NotificationListener
เป็นผู้สร้างการแจ้งเตือน ใช้ @TransactionalEventListener
ให้แจ้งเตือนถูกสร้างเมื่อบันทึกข้อมูลหลักสำเร็จแล้วเท่านั้น
6. **Template Method (กมลพร)** job รายวัน 3 ตัวมีขั้นตอนเหมือนกัน AbstractDailyJob
จึงกำหนดลำดับไว้ใน run() ที่เป็น final คลาสลูกเขียนแค่ "หาอะไร" และ "ทำอะไร" และใช้เวลาไทยเสมอ
7. **Command (พริ้มภัทร)** การเปลี่ยนบทบาทและระดับบัญชีถูกทำเป็น object ที่จำค่าเดิมไว้
UserCommandInvoker เก็บประวัติ 10 คำสั่งล่าสุดให้แอดมินย้อนกลับได้
8. **Chain of Responsibility (พริ้มภัทร)** การตรวจสอบข้อมูลสมัครสมาชิกแยกเป็น validator 4
ตัวต่อกันเป็นสาย ตรวจจากง่ายไปยาก และหยุดทันทีที่ขึ้นใดไม่ผ่าน

Class Diagram ของทุก pattern อยู่ในไฟล์ doc/design-patterns.md

บทที่ 5

การพัฒนาและทดสอบ

5.1 REST API และ Swagger

ระบบมี REST API 2 กลุ่ม ตอบเป็น JSON และใช้ Service ตัวเดียวกับหน้าเว็บ ผู้ใช้ต้องเข้าสู่ระบบก่อน

1. **ต้นไม้** /api/v1/plants ดูรายการแบบแบ่งหน้า (GET) ดูทีละต้น (GET /{id}) เพิ่ม (POST ตอบ 201) แก้ไข (PUT) และลบ (DELETE ตอบ 204) ถ้าเรียกต้นไม้ของคนอื่นจะได้ 404
2. **รายงานสุขภาพ** /api/v1/reports ดูรายการ (ผู้ใช้เห็นของตัวเอง แอดมินเห็นทั้งหมด) ดูทีละรายการ ส่งรายงาน (POST) ตอบรายงาน (PATCH /{id}/reply เฉพาะแอดมิน ไม่ใช่จะได้ 403) และลบรายงานของตัวเอง

Swagger UI สร้างอัตโนมัติด้วย springdoc-openapi เปิดได้ที่ /swagger-ui.html แสดงทุก endpoint พร้อมรูปแบบข้อมูลและทดลองเรียกได้จริง error ของ API ทุกตัวตอบเป็น JSON รูปแบบเดียวกันผ่าน GlobalExceptionHandler

5.2 การทดสอบ

ทีมเขียน Unit Test ด้วย JUnit 5, Mockito และ AssertJ โดยจำลอง Repository จึงทดสอบ business logic ได้โดยไม่ต้องใช้ฐานข้อมูลจริง และมี Integration Test 1 ตัวที่เปิดแอปพร้อมฐานข้อมูลจริง รวม 23 คลาส 111 test case แบ่งเป็นของสหรันท์ 23 ตัว กมลพร 54 ตัว มุกดา 20 ตัว พริ้มภัทร 13 ตัว และ Integration Test 1 ตัว ผลการรัน mvnw test เมื่อวันที่ 9 ตุลาคม 2569 ผ่านทั้งหมด 111 ตัว ไม่มี Failure และ Error

เทสต์ครอบคลุมทุก Design Pattern เช่น เทสต์ว่า Strategy ใช้ค่าตั้งต้นเมื่อพันธุไม่ได้ตั้งรอบ Iterator โยน exception เมื่อเรียกเกินวันสุดท้าย State ไม่ให้ต้นที่ตายแล้วเปลี่ยนสถานะ Command ย้อนคำสั่งได้ และ Observer ไม่ส่งแจ้งเตือนเมื่อบันทึกล้มเหลว รายละเอียด test case ทั้งหมดอยู่ในไฟล์ test/TEST_REPORT.md

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 111, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 41.501 s
[INFO] Finished at: 2569-10-09T22:30:57+07:00
[INFO] -----
PS D:\Tree_Management_Final\code> |
```

รูปที่ 5.1 ผลการรันเทสต์

5.3 Deployment และ CI/CD

ระบบ deploy เป็น Docker container บน Render ที่ <https://plantpal-owy8.onrender.com> Dockerfile สร้าง image 2 ชั้น คือ build ไฟล์ jar ด้วย Maven แล้วเหลือแค่ JRE กับ jar เพื่อให้ image เล็กและไม่รันด้วย root ค่าลับทั้งหมดส่งผ่าน environment variable ไม่อยู่ใน repo

CI/CD ใช้ GitHub Actions ทุกครั้งที่ push หรือเปิด Pull Request ระบบจะ build และรันเทสต์ทั้งหมดกับ PostgreSQL ชั่วคราว ถ้าเทสต์ไม่ผ่านจะไม่ deploy ถ้าผ่านและเป็นการ push เข้า develop จะสั่ง Render deploy เวอร์ชันใหม่อัตโนมัติ และมี workflow ปลุกเว็บทุก 10 นาทีช่วง 07:00–08:50 น. เพราะ Render free tier หลับเมื่อไม่มีคนใช้ งานแจ้งเตือนตอน 08:00 น. จึงทำงานได้

บทที่ 6

สรุป

ทีมพัฒนาระบบ PlantPal ได้ครบตามขอบเขต ผู้ใช้จัดการต้นไม้ ติดตามและบันทึกการดูแล ส่งรายงานสุขภาพ และได้รับการแจ้งเตือนอัตโนมัติ ระบบออกแบบตาม Layered Architecture และหลัก SOLID ใช้ Design Patterns 8 แบบแก้ปัญหาจริง มีเทสต์ 111 ตัวที่ผ่านทั้งหมด และ deploy อัตโนมัติขึ้น Render

6.1 ปัญหาที่พบ

1. โหลดข้อมูลต้นไม้ไม่ได้ในบางหน้า เพราะตั้ง open-in-view=false ทำให้อ่านข้อมูลแบบ LAZY นอก transaction ไม่ได้ แก้ด้วยการเขียน query แบบ join fetch ดึงข้อมูลที่ต้องใช้มาพร้อมกัน
2. โค้ดชนกันเมื่อทำงานพร้อมกัน 4 คน แก้ด้วยการแยก branch รายคน pull develop ก่อนเริ่มงานทุกครั้ง commit ทิ้งไฟล์ และให้เพื่อนตรวจผ่าน Pull Request
3. วันที่บนเว็บจริงคลาดเคลื่อนช่วงเช้ามีด เพราะ server บน Render ใช้เวลา UTC ส่วนบางหน้าใช้เวลาตามเครื่อง งานตั้งเวลาจึงกำหนดเวลาไทยไว้ชัดเจน
4. CI ไม่ผ่านทั้งที่ได้ถูก เพราะ Docker Hub จำกัดจำนวนครั้งดาวน์โหลด image ของฐานข้อมูลทดสอบ แก้ด้วยการรีนใหม่
5. เว็บเปิดครั้งแรกช้า เพราะ Render free tier กลับเมื่อไม่มีคนใช้ แก้ด้วย workflow ปลุกเว็บช่วงเช้า

6.2 แนวทางปรับปรุง

1. ตั้งเขตเวลาของทั้งแอปเป็นเวลาไทย ให้วันที่ถูกต้องตลอด 24 ชั่วโมง
2. เพิ่มการแจ้งเตือนผ่านอีเมลหรือ LINE โดยเพิ่ม listener ใหม่ตาม Observer ที่มีอยู่
3. รองรับการแนบรูปผ่าน REST API และเพิ่ม strategy ให้งานเช็คแคตและตรวจสอบสุขภาพ
4. เพิ่ม Integration Test ของ Controller และหน้าเว็บ และวัด code coverage
5. ให้ CI ดึง image จากแหล่งที่ไม่จำกัดจำนวนครั้ง และย้ายไป hosting แบบไม่หลัก